Extreme Programming e Lean Software Development

Processos de Software — Semana 3

Prof. Fernando · Engenharia de Software · UFRN · 2026.1

Leituras: Fowler — "Is Design Dead?" · Poppendieck — Lean Software Dev · Larman & Vodde — Lean Primer

PARTE 1 — Extreme Programming

XP: Contexto histórico

- Criado por **Kent Beck** em 1996 no projeto Chrysler C3
- Reação ao peso dos processos da época (cascata, RUP pesado)
- Nome provocativo: levar boas práticas ao **extremo**
  - Se revisão de código é bom → **pair programming** (revisão 100% do tempo)
  - Se testes são bons → **TDD** (testes antes do código)
  - Se integração frequente é boa → **integração contínua** (várias vezes ao dia)
  - Se design simples é bom → **YAGNI** (não implemente o que não precisa)

> O XP não inventou essas práticas — ele as levou ao extremo e mostrou que funcionam juntas como sistema.

Os 5 valores do XP

Valor	Significado
Comunicação	Todos falam entre si o tempo todo — sem documentos como proxy
Simplicidade	Faça a coisa mais simples que funciona — e só essa
Feedback	Ciclos curtos: testes (segundos), integração (minutos), releases (semanas)
Coragem	Coragem de jogar código fora, de mudar decisões, de dizer "não sei"
Respeito	Cada membro contribui e merece ser ouvido

> Esses valores não são decorativos — eles guiam as **decisões** quando as práticas parecem conflitar.

Design está morto? A pergunta de Fowler

Martin Fowler (2004) observou uma preocupação legítima:

> "Se o XP prega simplicidade e YAGNI, isso não significa abandono de design?"

A resposta dele:

- **Não.** O XP substitui **Big Up-Front Design** por **design evolutivo**
- Design continua existindo — mas emerge **ao longo** do desenvolvimento
- A diferença: o design é sustentado por **práticas habilitadoras** (*enabling practices*)

Sem essas práticas, design evolutivo se degrada em **"code and fix"** — que é o oposto de XP.

Práticas habilitadoras (Fowler)

O design evolutivo só funciona se **três práticas** estiverem presentes simultaneamente:

1. Testes automatizados

- Rede de segurança para mudanças
- Detectam regressões imediatamente
- Sem testes: medo de mudar → design congela

2. Refatoração contínua

- Mudar estrutura interna sem mudar comportamento
- Extrair classes, mover métodos, renomear, simplificar
- Sem refatoração: o design se degrada com o tempo

3. Integração contínua

- Integrar código várias vezes ao dia
- Testes rodam a cada integração
- Sem CI: mudanças acumulam, merge hell

> Fowler: essas práticas são **habilitadoras** — sem elas, as demais práticas falham. É um sistema interdependente.

A curva de custo de mudança

Argumento central de Fowler para o design evolutivo

Visão tradicional: custo de mudança cresce **exponencialmente** com o tempo

- → Logo, é melhor acertar tudo no início (Big Up-Front Design)

Argumento XP: com boas práticas técnicas, a curva pode ser **achatada**

- Testes automatizados → mudanças são seguras
- Refatoração → estrutura evolui sem quebrar
- CI → feedback imediato sobre impacto

Se a curva é plana, **não há penalidade** em decidir depois — e você decide com **mais informação**.

> Isso conecta diretamente com o princípio Lean: **"Decidir o mais tarde possível"**

YAGNI — You Aren't Gonna Need It

- Não implemente funcionalidades **até que sejam necessárias**
- Combate a **especulação** sobre requisitos futuros
- Funcionalidades não usadas são **desperdício** (conceito Lean!)

Exemplo prático:

Tentação	YAGNI diz...
"Vamos fazer a API aceitar XML também, caso alguém precise"	Não. Implemente JSON. Se pedirem XML, implemente XML.
"Melhor criar uma abstração genérica para todos os bancos"	Não. Use PostgreSQL. Se mudar de banco, refatore.
"Vou adicionar cache desde o início"	Não. Meça primeiro. Otimize depois.

> YAGNI não é contra planejamento — é contra **implementação especulativa**.

TDD: Test-Driven Development

O ciclo Red → Green → Refactor

1. **RED** — Escreva um teste que falha (o comportamento desejado ainda não existe)
2. **GREEN** — Implemente o mínimo para o teste passar
3. **REFACTOR** — Melhore a estrutura sem mudar o comportamento

Por que TDD guia o design? (Fowler)

- Você pensa na **interface** antes da implementação
- Código difícil de testar → sinal de **design ruim**
- O teste É a especificação — documentação executável

> Na Lista 2 de Web II, vocês viram isso: testes chamam `NewRouter()` antes do código existir — TDD natural.

Pair Programming

- Duas pessoas, um computador, um problema
- **Driver**: escreve o código
- **Navigator**: revisa, pensa no design, identifica problemas

Benefícios comprovados:

- Revisão de código em **tempo real** (não depois de dias)
- Compartilhamento de conhecimento — reduz **bus factor**
- Menos defeitos chegam ao repositório
- Integração social — mais do que divisão de tarefas

O que pair programming não é:

- Não é um programador trabalhando e outro assistindo
- Não é mentoria (embora possa incluir)
- Os papéis se alternam frequentemente

Integração Contínua (CI)

- Integrar código ao branch principal **várias vezes ao dia**
- Cada integração dispara **build + testes automatizados**
- Problemas são detectados em **minutos**, não em semanas

O contrato da CI (Fowler, 2024):

1. Mantenha um repositório único
2. Automatize o build
3. Faça o build se auto-testar
4. Todo commit roda no servidor de CI
5. **Conserte o build quebrado imediatamente** — é prioridade máxima
6. Mantenha o build rápido

> CI é a prática que **une** testes, refatoração e design evolutivo. Sem CI, as outras práticas se desconectam.

XP como sistema: as práticas se reforçam

```
Pair Programming ← → Code Review contínuo
     ⬍                      ⬍
TDD ← → Design Evolutivo ← → Refatoração
     ⬍                      ⬍
CI ← → Feedback Rápido ← → Releases Curtos
     ⬍                      ⬍
YAGNI ← → Simplicidade ← → Coragem de Mudar
```

> Fowler: "O XP não funciona se você adota **apenas algumas** práticas. As práticas se habilitam mutuamente."

O sistema inteiro depende de **disciplina técnica** — não de processos burocráticos.

PARTE 2 — Lean Thinking

O que é Lean Thinking?

- Sistema desenvolvido pela **Toyota** (Toyota Production System → Toyota Way)
- Termo popularizado por pesquisadores do **MIT** nos anos 90
- Não é um conjunto de ferramentas — é uma **cultura e filosofia de gestão**

O princípio central:

> **"Assista o bastão, não os corredores."**

- O "bastão" = **fluxo de valor** ao cliente
- Os "corredores" = pessoas/recursos individuais
- Gestão tradicional otimiza os corredores (utilização 95%!)
- Lean otimiza o **bastão** (velocidade de entrega de valor)

Larman & Vodde, Lean Primer, 2009

Os dois pilares do Lean

Respeito pelas Pessoas

- **Não incomode seu cliente** (interno ou externo)
- "Desenvolver pessoas, depois produtos"
- Gestores são **professores**, não diretores
- Times decidem **como** melhorar
- Parcerias de longo prazo baseadas em confiança

Melhoria Contínua (Kaizen)

- **Go See** (Genchi Genbutsu) — vá ao local do trabalho real
- Kaizen — pequenas melhorias, sem parar, por todos
- Desafio da perfeição — insatisfação saudável com o status quo
- Fluxo — reduzir lotes, filas, tempo de ciclo

> Toyota CEO Watanabe: *"A raiz do Toyota Way é estar insatisfeito com o status quo."*

Toyota Way 2001 — pilares conforme descrição interna

O que Lean NÃO é

Equívoco	Realidade
Lean = Kanban	Kanban é uma **ferramenta**. Lean é cultura + pilares + 14 princípios
Lean = eliminar desperdício	Desperdício é uma consequência com causas raíz. Identifique a causa e corrija o quanto antes
Lean = Lean Six Sigma	Toyota vê Lean Six Sigma como ferramentas de Six Sigma, não Lean real
Lean = fazer mais com menos	Lean é sobre **entregar mais valor**, não cortar custos cegamente
Lean = implementar rápido	Adoção de Lean leva **anos** — requer mudança cultural profunda

> *"Cargo cult lean"* = adotar ferramentas sem entender a filosofia. É como reduzir democracia a votação.

Larman & Vodde, Lean Primer, p. 4-6

Gestores-professores: a fundação

- Na Toyota, gestores devem ser **mestres** no trabalho que gerenciam
- Ditado interno: *"Meu gerente faz meu trabalho melhor que eu"*
- Novos funcionários passam **meses** em educação antes de trabalhar
- Executivos crescem **de dentro**, com anos de prática e mentoria
- Lema interno: **"Good Thinking, Good Products"** (Bom Pensamento, Bons Produtos)

O que gestores fazem diferente na Toyota:

Gestor tradicional	Gestor-professor (Lean)
Cobra relatórios, define metas	Vai ao **Gemba** observar o trabalho real
Decide por cima	**Ensina** o time a pensar e resolver problemas
Especialista em gestão genérica	Mestre na área técnica que gerencia
Foca em utilização de recursos	Foca em **fluxo de valor**

Lean Primer, p. 10-12

Go See (Genchi Genbutsu)

- **Ir ao local real** do trabalho para ver com os próprios olhos
- Não confiar apenas em relatórios, dashboards, números

> Taiichi Ohno: *"Não olhe com seus olhos, olhe com seus pés. Pessoas que só olham números são as piores de todas."*

Em software, Go See significa:

- Sentar com o desenvolvedor e **ver o código sendo escrito**
- Assistir a um deploy e ver **onde trava**
- Participar de um code review como observador
- Usar o produto como um **usuário real**

> Não basta estar no mesmo prédio — Gemba significa **"respirar o mesmo ar"** que quem faz o trabalho.

Lean Primer, p. 14-15

Kaizen: mais que "melhoria contínua"

As 3 etapas do Kaizen:

1. **Dominar um método padronizado** — criado pelo time, não imposto de cima
2. **Experimentar** até encontrar forma melhor
3. **Repetir para sempre**

Pontos cruciais (do Lean Primer):

- *Standardized work* ≠ conformidade a padrões centralizados
- O padrão é criado **pelo time** e **sempre evolui** — *yokoten* (espalhar lateralmente)
- Ohno: *"Se o trabalho padronizado ficou igual por um mês inteiro, vocês não estão ganhando seu salário"*
- Fracasso de experimentos é OK — o único fracasso é **não experimentar**

> Kusunoki (VP Toyota): *"Gestores não punem quem tenta e erra. Punem quem não tenta."*

Lean Primer, p. 16-18

5 Porquês (5 Whys)

Ferramenta para encontrar a **causa raiz** de problemas:

```
Problema: Build quebrou na CI

Por quê? → Teste de integração falhou
Por quê? → Serviço externo estava fora do ar
Por quê? → Teste depende de serviço real, não de mock
Por quê? → Equipe não sabe criar mocks para serviços HTTP
Por quê? → Não houve mentoria sobre técnicas de teste
          → AÇÃO: sessão de pair programming sobre mocks
```

Regras importantes:

- "5" não é número mágico — é "cave fundo"
- Deve ser feito no **Gemba** (no local do problema), não em reunião
- Pode gerar grafo de causas (diagrama de Ishikawa/espinha de peixe)
- Conecta com Go See: sem ver os fatos, as respostas são chutes

Lean Primer, p. 19

Valor e Desperdício (Muda)

Definições fundamentais:

- **Valor** = momentos que criam o produto pelo qual o cliente paga
- **Desperdício** = tudo que consome recursos mas não agrega valor

Taxa de valor:

```
taxa_de_valor = tempo_em_atividades_de_valor / tempo_total (conceito_ao_caixa)
```

Em desenvolvimento de software, taxas de valor acima de **7% são raras**.

> 93% do tempo em desenvolvimento é desperdício. A estratégia Lean: em vez de melhorar os 7% de valor, **eliminar o desperdício** dos 93%.

Lean Primer, p. 20

10 tipos de desperdício em software

#	Desperdício	Exemplo em software
1	**Superprodução**	Features que ninguém usa
2	**Espera**	Aguardando aprovação, code review, ambiente
3	**Transferências** (handoff)	Analista → Dev → QA — cada handoff perde contexto
4	**Processos extras**	Checklists de qualidade impostos que não agregam
5	**Trabalho parcial** (WIP)	Código escrito mas não integrado, não testado, não entregue
6	**Troca de contexto**	Dev alocado em 3 projetos simultaneamente
7	**Defeitos**	Testes e correções no final do ciclo
8	**Talento subutilizado**	Dev fazendo só o que o job title diz
9	**Informação dispersa**	Requisitos espalhados em 5 documentos e 3 ferramentas
10	**Pensamento ilusório**	"Estamos atrasados, mas vamos recuperar"

Lean Primer, p. 21-22 + Toyota Way + Poppendieck

3 fontes de desperdício: Mura, Muri, Muda

Fonte	Significado	Exemplo em software	Resolução
Mura (variabilidade)	Ciclos irregulares, tamanhos de lote inconsistentes	Sprints de tamanhos variáveis, interrupções	Cadência, timeboxes, decomposição
Muri (sobrecarga)	Pressão insustentável sobre pessoas	Overtime para deadlines arbitrários, especialista gargalo	Limitar WIP, cross-training, descope
Muda (ações sem valor)	Os 10 tipos anteriores	Handoff, espera, WIP, defeitos	Kaizen, eyes for waste

> Toyota ensina que **mura** e **muri** são frequentemente **causas raiz** que geram **muda**. Trabalhadores sobrecarregados produzem mais defeitos.

Lean Primer, p. 23-24

Os 7 princípios de Lean Software Development

Mary e Tom Poppendieck (2003) — leitura do quiz

#	Princípio	Ideia central
1	**Eliminar desperdício**	Tudo que não agrega valor ao cliente é candidato a eliminação
2	**Amplificar aprendizado**	Ciclos curtos de feedback, experimentação, iteração
3	**Decidir o mais tarde possível**	Adiar decisões irreversíveis até ter mais informação
4	**Entregar o mais rápido possível**	Ciclos curtos → feedback → melhores decisões
5	**Empoderar a equipe**	Quem faz o trabalho decide como fazer
6	**Construir integridade**	Qualidade desde o início, não inspecionada no final
7	**Ver o todo**	Otimizar o sistema, não partes isoladas

> Princípios 3 e 4 parecem contraditórios, mas são **complementares**: entregas rápidas dão feedback que permite decidir melhor.

Os 14 princípios do Toyota Way (resumo)

#	Princípio	#	Princípio
1	Decisões de **longo prazo**	8	Tecnologia **testada**
2	Mover em direção ao **fluxo**	9	Líderes **de dentro**
3	**Sistemas puxados** (pull)	10	Pessoas **excepcionais**
4	**Nivelar** a carga (heijunka)	11	Parcerias de **longo prazo**
5	Cultura de **parar e corrigir**	12	**Go See**
6	Padrões **evoluídos pelo time**	13	Decisão por **consenso**
7	**Gestão visual** simples	14	**Aprendizado contínuo** (kaizen)

> Fujio Cho (presidente Toyota): *"O importante é ter todos os elementos juntos como sistema. Deve ser praticado todos os dias de forma consistente."*

The Toyota Way (Liker, 2004)

Fluxo e a metáfora do lago e das rochas

Princípio: trabalhar em lotes pequenos e ciclos curtos

A metáfora Toyota:

- **Lago** = tamanho do lote / tempo de ciclo
- **Rochas** = problemas e ineficiências ocultos
- Água alta (lotes grandes) → rochas invisíveis
- Água baixa (lotes pequenos) → rochas aparecem

Em software:

- Release a cada 18 meses → problemas de integração e teste ficam ocultos
- Release a cada 2 semanas → práticas ineficientes se tornam **insuportáveis**
  - → Isso **força** a melhoria (automação, CI, testes)

> A dor do ciclo curto é o motor da melhoria contínua.

Lean Primer, p. 27-29

Sistemas puxados (Pull) vs empurrados (Push)

Push (empurrar)

- Produzir especulativamente
- Criar inventário "por via das dúvidas"
- Grandes lotes de documentos, designs, features
- Em software: escrever specs de 200 features antes de codificar

Pull (puxar)

- Produzir **sob demanda** do cliente
- Zero inventário como meta
- Em software: implementar quando **solicitado**, não antes
- Pull expõe defeitos rapidamente (consumo imediato)

Conexão com XP:

- **YAGNI** é Pull aplicado ao design: só implemente quando puxado pela necessidade real
- **Decidir o mais tarde possível** = Pull para decisões arquiteturais

Lean Primer, p. 30-31

Gestão Visual

- Toyota enfatiza ferramentas visuais **grandes, físicas, coloridas**
- O oposto de informações escondidas em planilhas e dashboards
- **Kanban** (看板) = sinal visual + cartão → puxa trabalho
- **Andon** = sinal luminoso de problema → parar e corrigir

Em software:

Prática visual	Finalidade
Quadro Kanban físico	Ver filas, WIP, gargalos
Build radiator (tela vermelha/verde)	Status do CI visível para todos
Burndown chart na parede	Progresso do sprint
Post-its com tarefas	Sentir fisicamente o tamanho do trabalho

> Filas **invisíveis** (bits no computador) não geram urgência. Tokens **físicos** criam "olhos para filas".

Lean Primer, p. 31-34

Parar e corrigir (Stop & Fix + Jidoka)

- Na Toyota, **qualquer pessoa** pode parar a linha de produção ao ver um defeito
- Não é só corrigir: é aplicar **5 Whys** para encontrar a causa raiz
- **Jidoka** = automação com toque humano: máquinas que detectam falhas e param sozinhas

Em software:

- Build quebrou na CI? → **Prioridade máxima** é consertar (não ignorar)
- Bug em produção? → Não só hotfix: entender **por que** o bug passou pelos testes
- Teste flaky (intermitente)? → Não desabilitar: investigar a causa

> Isso conecta diretamente com CI no XP: o build quebrado é o equivalente do "andon" — sinal vermelho que exige ação imediata.

XP e Lean: convergências

Conceito	XP	Lean
Feedback rápido	TDD + CI + releases curtos	Ciclos curtos + pull
Simplicidade	YAGNI	Eliminar desperdício
Qualidade embutida	TDD + refatoração	Construir integridade + Jidoka
Empoderamento	Equipe decide práticas	Times evoluem seus próprios padrões
Aprendizado	Pair programming + retrospectivas	Kaizen + Go See + 5 Whys
Decisões tardias	Design evolutivo (Fowler)	Decidir o mais tarde possível

> Fowler (2004) e Poppendieck (2003) convergem: **ciclos curtos de feedback e aprendizado contínuo** são o mecanismo central para lidar com incerteza em software.

Lean em software: práticas concretas

Princípio Lean	Prática em software
Fluxo	CI/CD, deploy contínuo
Pequenos lotes	Sprints curtos, feature flags, trunk-based dev
Pull	Kanban com WIP limits
Gestão visual	Quadro Kanban, build radiator
Eliminar desperdício	Automatizar testes, reduzir handoffs
Cadência	Timeboxes regulares (2 semanas)
Parar e corrigir	Build vermelho = prioridade #1
Go See	Pair programming, mob programming
Kaizen	Retrospectivas com ações concretas

> Muitas práticas que vocês já viram (CI, TDD, pair programming) **são** práticas Lean aplicadas a software.

Lean Product Development

Objetivo: "Aprender mais que a concorrência"

- Engenheiros-chefes **empreendedores** (técnico + negócio em uma pessoa)
- Design baseado em **conjuntos** (*set-based*): explorar múltiplas alternativas em paralelo
- **Reutilizar conhecimento**: mentoria, wikis, registros de experimentos
- Salas de equipe **multifuncionais** sem paredes, com gestão visual
- **Cadência** regular (takt time para desenvolvimento)

Resultado Toyota:

- Projetos concluídos em **5 meses** (vs. 12 da concorrência)
- Menor razão desenvolvimento/vendas do setor automotivo
- Engenheiros permanecem **técnicos por anos** (não viram gerentes cedo)

Lean Primer, p. 34-41

Erros comuns na adoção do Lean

Armadilha	Por quê é um erro
Adotar Kanban sem mudar cultura	Kanban é ferramenta — sem kaizen mindset, vira burocracia visual
Gestores delegam Lean para consultores	Fundação do Lean = gestores-professores que vivem os princípios
Padronização centralizada	Toyota: padrões são criados **pelo time** e evoluem localmente
Foco apenas em eliminar desperdício	Desperdício é consequência — atacar variabilidade e sobrecarga
Copiar práticas sem entender contexto	"Cargo cult lean" — rituais sem substância

> Toyota renomeou o livro interno para **"Toyota Way 2001"** (não "Toyota Way Final") para enfatizar: **não existe processo definitivo**.

Lean Primer, p. 24-25

Atividade Prática (15 min)

Em grupo (3–4 pessoas):

1. **Identifiquem** um desperdício no processo de desenvolvimento do projeto de vocês (ou de um projeto anterior)
   - Use a lista de 10 tipos de muda como referência
2. Apliquem os **5 Porquês** para encontrar a causa raiz
3. Proponham **uma melhoria** usando o ciclo PDCA:
   - **Plan**: o que vamos mudar?
   - **Do**: como vamos experimentar?
   - **Check**: como vamos medir se melhorou?
   - **Act**: como incorporar a melhoria?

Recapitulando — XP

- Design evolutivo sustentado por **práticas habilitadoras** (Fowler)
- Testes automatizados + refatoração + CI = tripé do design evolutivo
- TDD guia o design pela interface — não é "só" técnica de teste
- YAGNI: não implemente o que não precisa (= eliminar desperdício!)
- Práticas se reforçam mutuamente — XP é um **sistema**, não um menu

Recapitulando — Lean

- **Dois pilares**: Respeito pelas Pessoas + Melhoria Contínua
- Fundação: **gestores-professores** que vivem e ensinam os princípios
- Go See, Kaizen, 5 Whys — práticas de melhoria contínua
- 7 princípios Poppendieck para software, 14 princípios Toyota Way
- Muda/Mura/Muri — 3 fontes de desperdício
- Fluxo, pull, lotes pequenos, gestão visual — princípios operacionais
- O bastão, não os corredores — otimize o sistema, não as partes

Referências

- **Fowler, M.** (2004). *Is Design Dead?* — martinfowler.com
- **Poppendieck, M. & T.** (2003). *Lean Software Development: An Agile Toolkit*. Addison-Wesley
- **Larman, C. & Vodde, B.** (2009). *Lean Primer*. leanprimer.com (CC)
- **Liker, J.** (2004). *The Toyota Way*. McGraw-Hill
- **Ohno, T.** (1988). *Toyota Production System*. Productivity Press
- **Fowler, M.** (2024). *Continuous Integration*. martinfowler.com
- **Beck, K.** (1999). *Extreme Programming Explained*. Addison-Wesley